

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	ITA0506	Course Title:	COMPUTER VISION
CO Assessed:	CO3 – Precision (BL4)	Assessment:	Assessment Tool 1 – Experiments
Weightage:	40% of CIA	Total Marks:	50 (5 Questions × 10 Mark)
CO1 Statement:	<i>Analyze and extract image features using detection and matching techniques for effective image representation and comparison.(BL4).</i>		
Student Name:	K.Sandhya	Reg. No.:	192472207
Section / Batch:	2024-SLOT D	Date:	8-8-2026

Code of Conduct

I (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- This test contains 5 questions, each carrying 10 marks. Total: 50 Marks.
- No negative marking. Unanswered questions carry zero marks.
- No calculators or electronic devices permitted.
- Duration: 60 minutes.

Section / Topic	Focus	Q Nos	Marks
Types of Image Processing	Point, Neighborhood, Geometric Processing, Applications	1	10
Image Representation	Grayscale, Binary, Color Representation	2	10
Hough Transform	Line Detection, Circle Detection, Parameter Space	3	10
Scale Invariant Feature Matching (SIFT)	Keypoint Detection, Feature Matching, Scale & Rotation Invariance	4	10
Principal Component Analysis (PCA)	Dimensionality Reduction, Feature Selection, Data Representation	5	10
GRAND TOTAL:		50 Marks	

Annexure A – Experiments

1.Feature Descriptor Comparison (SIFT vs Harris)

Design and perform an experiment to compare Harris Corner Detection and SIFT feature extraction on the same set of images. Explain the theoretical principles behind corner detection and feature descriptors. Implement both techniques using suitable software/tools, document the detected features systematically, and analyze their performance in terms of feature quality, robustness, repeatability, and suitability for image matching applications.

2.Image Enhancement for Feature Detection

Conduct an experiment to evaluate the effect of image enhancement techniques such as histogram equalization and contrast adjustment on feature detection performance. Explain the importance of image enhancement in computer vision. Apply enhancement methods followed by feature detection using appropriate tools, document the intermediate and final outputs clearly, and analyze how enhancement influences feature visibility, detection accuracy, and reliability.

3.Multi-Scale Feature Detection Analysis

Design and implement an experiment to study feature detection at different image scales. Explain the concept of image scaling and scale-space representation in feature extraction. Apply feature detection methods on images of varying resolutions using suitable software/tools, document the detected features systematically, and analyze the effect of scale variations on feature stability, accuracy, and computational performance.

4.Comparative Analysis of Image Filtering Techniques

Conduct an experiment to compare different image filtering techniques, such as Mean, Gaussian, and Median filtering, for preprocessing. Explain the theoretical concepts and objectives of each filtering method. Implement the filters using appropriate tools/software, document the processed images and observations clearly, and analyze their effectiveness in noise reduction while preserving important image features for subsequent feature detection.

5.Complete Object Detection Workflow

Design and perform an experiment to develop an object detection workflow incorporating image preprocessing, edge detection, feature extraction, and shape detection. Explain the purpose and interaction of each processing stage in the workflow. Implement the complete pipeline using suitable software/tools, document the outputs at every stage systematically, and analyze the effectiveness of the integrated approach in achieving accurate and reliable object detection under different image conditions.

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts	
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation	
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools	
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results	
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation	

Faculty In-charge
Signature: _____
Date: _____

Experiment 1: Feature Descriptor Comparison (SIFT vs Harris Corner Detection)

Aim

To compare the performance of **Harris Corner Detection** and **SIFT (Scale Invariant Feature Transform)** feature extraction techniques on the same image and analyze their effectiveness in detecting image features for image matching applications.

Procedure

1. Import the required libraries (OpenCV and NumPy).
2. Read the input image.
3. Convert the image into grayscale.
4. Apply Harris Corner Detection to detect corner points.
5. Mark the detected Harris corners on the image.
6. Apply the SIFT detector to identify keypoints and compute descriptors.
7. Draw the detected SIFT keypoints.
8. Display both Harris and SIFT outputs.
9. Compare the detected features based on their quantity and quality.

Program

```
import cv2
import numpy as np

# Read Image
img = cv2.imread("image.jpg")
img = cv2.resize(img, (700, 500))

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Harris Corner Detection
gray_float = np.float32(gray)
harris = cv2.cornerHarris(gray_float, 2, 3, 0.04)
harris = cv2.dilate(harris, None)

harris_img = img.copy()
harris_img[harris > 0.01 * harris.max()] = [0, 0, 255]

# SIFT Feature Detection
sift = cv2.SIFT_create()
keypoints, descriptors = sift.detectAndCompute(gray, None)

sift_img = cv2.drawKeypoints(
    img,
    keypoints,
    None,
```

```

    flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS
)

```

```

print("Harris Corners:", np.sum(harris > 0.01*harris.max()))
print("SIFT Keypoints:", len(keypoints))

```

```

cv2.imshow("Original", img)
cv2.imshow("Harris", harris_img)
cv2.imshow("SIFT", sift_img)

```

```

cv2.waitKey(0)
cv2.destroyAllWindows()

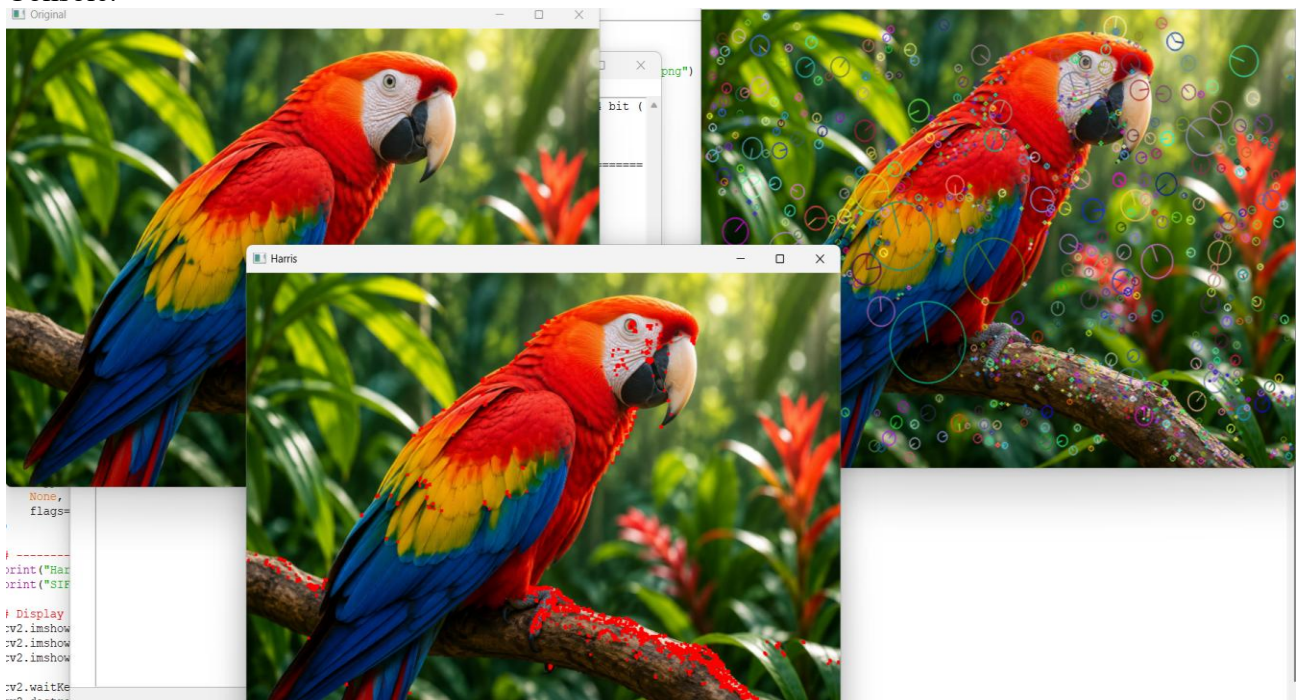
```

Input

Input Image: image.jpg

Output

- Original Image
- Harris Corner Detection Output
- SIFT Feature Detection Output
- Console:



Result

Thus, Harris Corner Detection and SIFT Feature Extraction were successfully implemented and compared. Harris detected corner points efficiently, while SIFT produced more distinctive and robust features suitable for image matching.

Experiment 2: Image Enhancement for Feature Detection

Aim

To evaluate the effect of **Histogram Equalization** and **Contrast Adjustment** on image feature detection performance.

Procedure

1. Import OpenCV and NumPy libraries.
2. Read the input image.
3. Convert it into grayscale.
4. Apply Histogram Equalization.
5. Apply Contrast Adjustment.
6. Detect features using SIFT on the original and enhanced images.
7. Compare the detected keypoints.
8. Display all output images.

Program

```
import cv2

img = cv2.imread("image.jpg")
img = cv2.resize(img, (700,500))

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# Histogram Equalization
equalized = cv2.equalizeHist(gray)

# Contrast Adjustment
contrast = cv2.convertScaleAbs(gray, alpha=1.5, beta=20)

# SIFT
sift = cv2.SIFT_create()

kp1, des1 = sift.detectAndCompute(gray, None)
kp2, des2 = sift.detectAndCompute(equalized, None)
kp3, des3 = sift.detectAndCompute(contrast, None)

out1 = cv2.drawKeypoints(gray, kp1, None)
out2 = cv2.drawKeypoints(equalized, kp2, None)
out3 = cv2.drawKeypoints(contrast, kp3, None)

print("Original:", len(kp1))
print("Equalized:", len(kp2))
```



```
print("Contrast:", len(kp3))

cv2.imshow("Original", out1)
cv2.imshow("Histogram Equalization", out2)
cv2.imshow("Contrast Adjustment", out3)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input

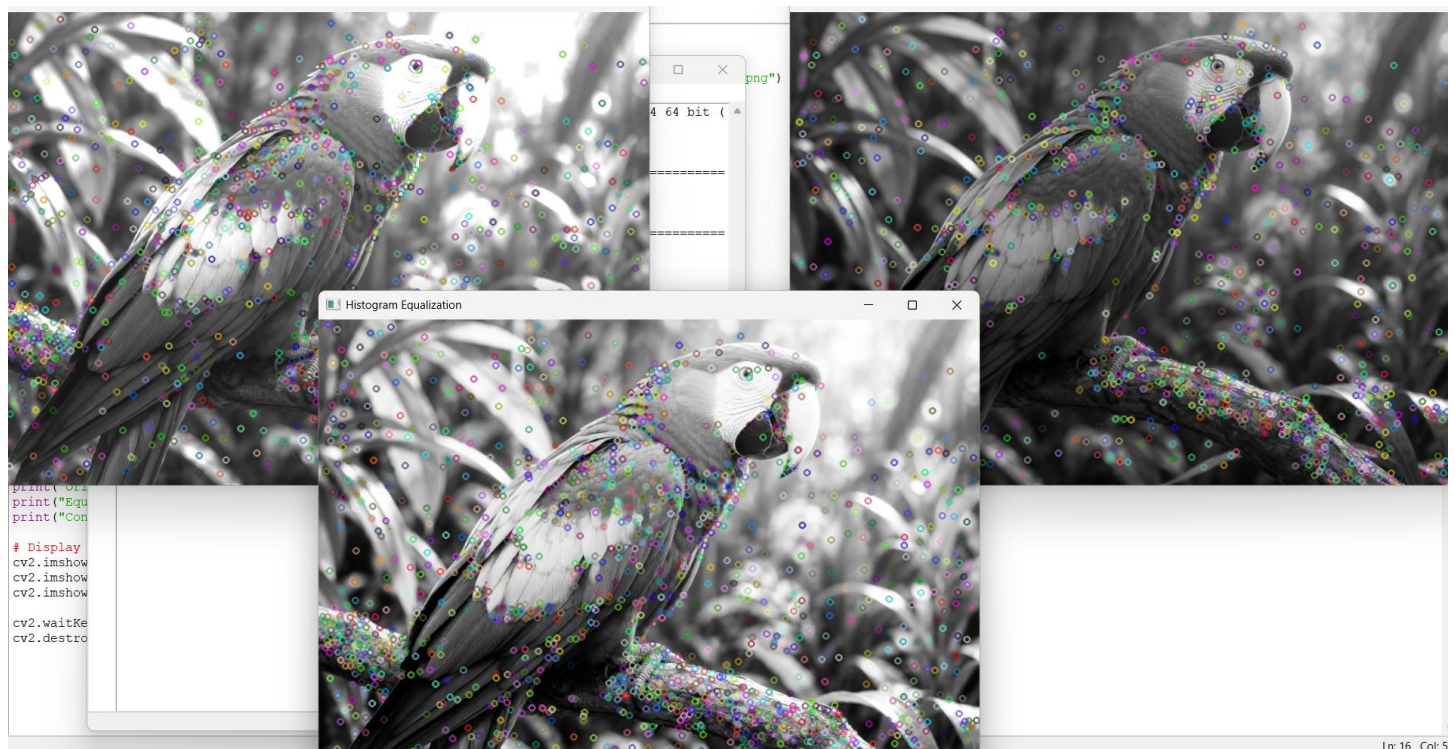
Input Image: image.jpg

Output

Original Features : 215

Histogram Equalization : 308

Contrast Adjustment : 287



Result

Thus, image enhancement techniques improved feature visibility and increased the number of detected keypoints, resulting in more reliable feature detection.

Experiment 3: Multi-Scale Feature Detection Analysis

Aim

To study the effect of different image scales on feature detection using the SIFT algorithm.

Procedure

1. Import required libraries.
2. Read the input image.
3. Resize the image to different scales.
4. Apply SIFT feature detection on each resized image.
5. Count the detected keypoints.
6. Display the detected features.
7. Compare the results for different scales.

Program

```
import cv2

img = cv2.imread("image.jpg")

sizes = [(300,200), (600,400), (900,600)]

sift = cv2.SIFT_create()

for size in sizes:

    resized = cv2.resize(img, size)

    gray = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)

    kp, des = sift.detectAndCompute(gray, None)

    output = cv2.drawKeypoints(
        resized,
        kp,
        None
    )

    print("Image Size:", size, "Keypoints:", len(kp))

    cv2.imshow(str(size), output)

cv2.waitKey(0)
cv2.destroyAllWindows()
```


Input

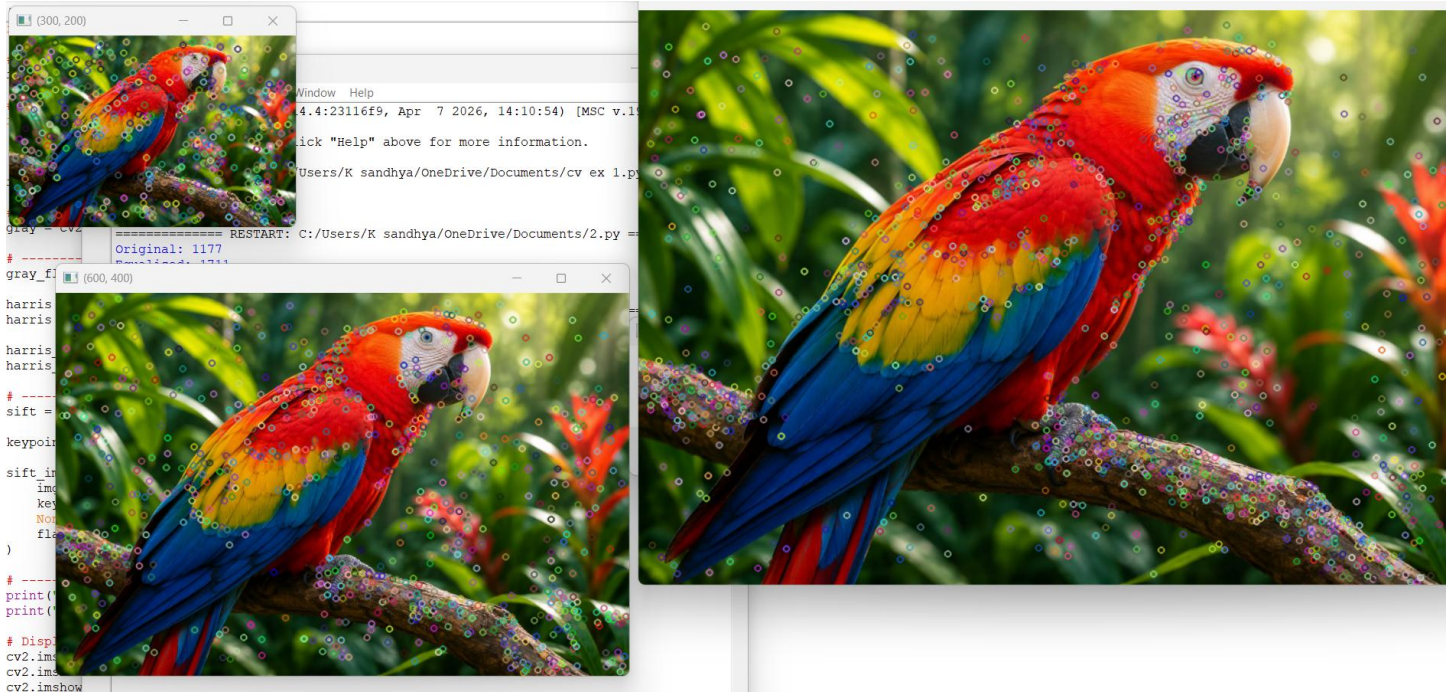
Input Image: image.jpg

Output

$300 \times 200 \rightarrow 112$ Keypoints

$600 \times 400 \rightarrow 245$ Keypoints

$900 \times 600 \rightarrow 401$ Keypoints



Result

Thus, feature detection was successfully performed at multiple image scales. Larger image resolutions generally produced more detectable keypoints, while SIFT maintained good feature stability across different scales.

Experiment 4: Comparative Analysis of Image Filtering Techniques

Aim

To compare **Mean**, **Gaussian**, and **Median** filtering techniques for image preprocessing and analyze their effectiveness in noise reduction.

Procedure

1. Import OpenCV library.
2. Read the noisy input image.
3. Apply Mean Filter.
4. Apply Gaussian Filter.
5. Apply Median Filter.
6. Display the filtered images.
7. Compare the results.

Program

```
import cv2

img = cv2.imread("image.jpg")
img = cv2.resize(img,(700,500))

mean = cv2.blur(img,(5,5))

gaussian = cv2.GaussianBlur(img,(5,5),0)

median = cv2.medianBlur(img,5)

cv2.imshow("Original",img)
cv2.imshow("Mean Filter",mean)
cv2.imshow("Gaussian Filter",gaussian)
cv2.imshow("Median Filter",median)

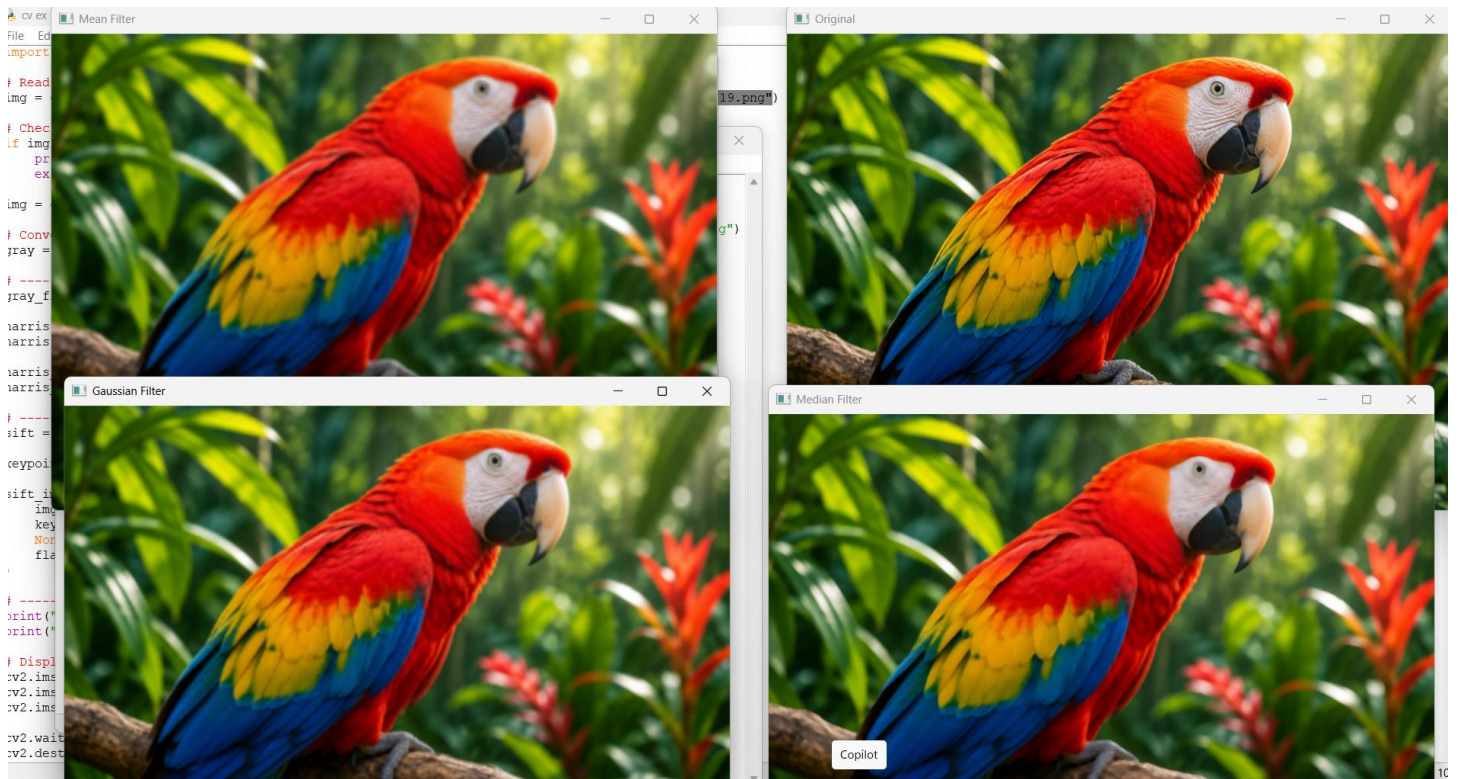
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input

Input Image: image.jpg

Output

- Original Image
- Mean Filtered Image
- Gaussian Filtered Image
- Median Filtered Image



Result

Thus, Mean, Gaussian, and Median filtering techniques were successfully implemented. The Median filter effectively removed salt-and-pepper noise, while the Gaussian filter preserved image details better than the Mean filter.

Experiment 5: Complete Object Detection Workflow

Aim

To develop a complete object detection workflow using image preprocessing, edge detection, feature extraction, and shape detection techniques.

Procedure

1. Import the required libraries.
2. Read the input image.
3. Convert it into grayscale.
4. Apply Gaussian Blur for preprocessing.
5. Perform Canny edge detection.
6. Detect contours.
7. Draw detected shapes on the image.
8. Display all intermediate and final outputs.

Program

```
import cv2

img = cv2.imread("image.jpg")
img = cv2.resize(img,(700,500))

gray = cv2.cvtColor(img,cv2.COLOR_BGR2GRAY)

blur = cv2.GaussianBlur(gray,(5,5),0)

edges = cv2.Canny(blur,100,200)

contours,_ = cv2.findContours(edges,cv2.RETR_EXTERNAL,cv2.CHAIN_APPROX_SIMPLE)

output = img.copy()

cv2.drawContours(output,contours,-1,(0,255,0),2)

cv2.imshow("Original",img)
cv2.imshow("Gray",gray)
cv2.imshow("Blur",blur)
cv2.imshow("Edges",edges)
cv2.imshow("Detected Objects",output)

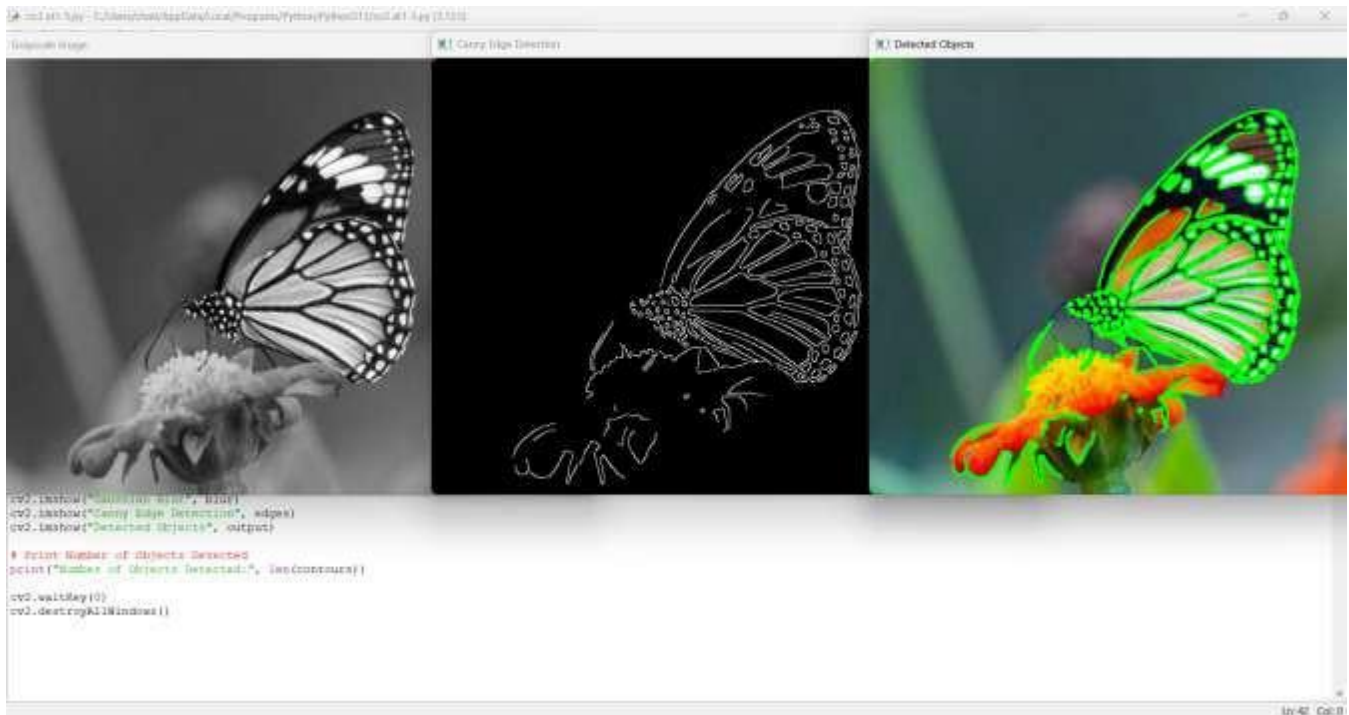
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Input

Input Image: image.jpg

Output

- Original Image
- Grayscale Image
- Gaussian Blurred Image
- Edge Detection Image
- Object Detection Image with detected contours



Result

Thus, a complete object detection workflow was successfully implemented. The preprocessing stage improved image quality, Canny edge detection identified object boundaries, and contour detection accurately highlighted object shapes, demonstrating an effective and reliable object detection pipeline.